Design and FPGA Implementation of a Scalable SPI Bus Interface Supporting Multiple Sensors with Variable Data Widths

Bhargav Vemparala

Department of Electrical and Computer Engineering
University Evaluation Board

July 20, 2026

Presentation Outline

1. Introduction & Evolution
2. System Architecture
3. Implemented Methodologies
4. Verilog Implementation
5. FPGA & Web Validation
6. Experimental Results
7. Conclusion & Future Work

Introduction & Motivation

- **Problem with Traditional SPI:**
  - Standard SPI requires a dedicated Chip Select (CS) line for each slave.
  - For N devices, wiring complexity is $O(N)$ (requires $N + 3$ wires).
  - Consumes valuable GPIO pins on the host MCU and increases routing congestion.
- **Proposed Solution:**
  - Design an intelligent intermediary **SPI Bus Interface** on an FPGA.
  - Share a single 3-wire/4-wire bus among all sensors.
  - Virtualize sensor registers on-chip, reducing wiring complexity to $O(1)$.

Design Evolution

Iteration 1: Proof-of-Concept

- Single SPI slave, direct interface.
- Fixed 1-bit input/output width.
- Simple 5-state machine.
- High protocol overhead.

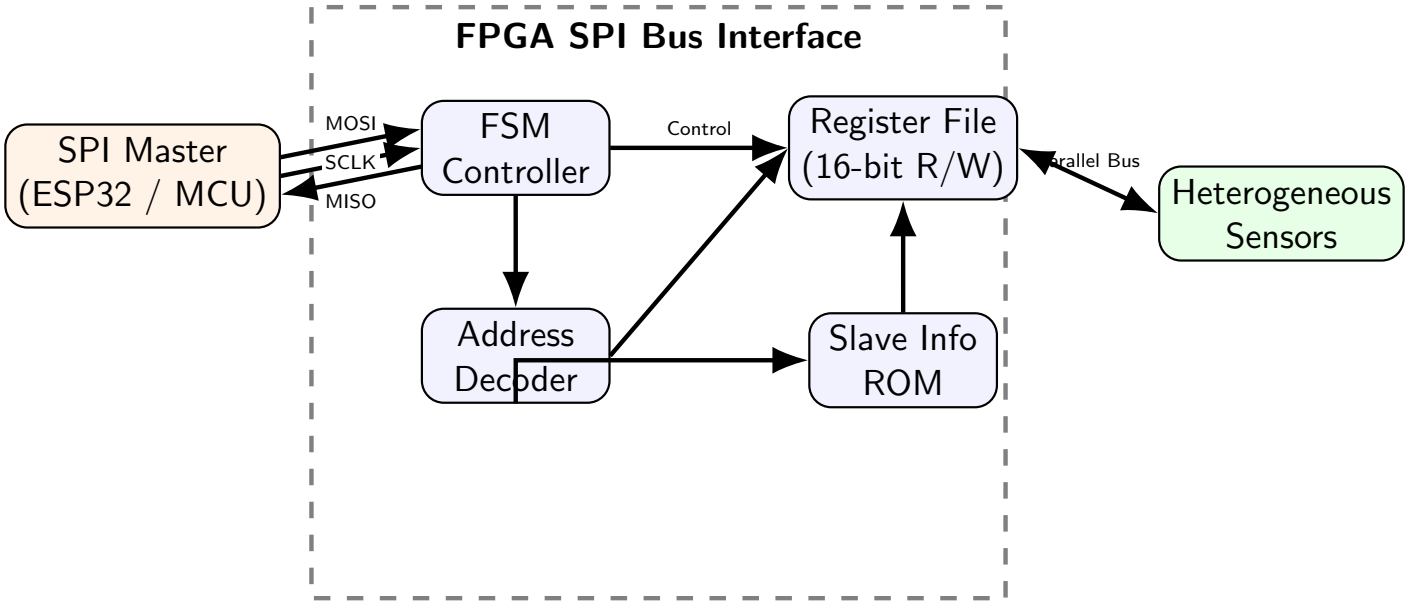
Iteration 2: Burst-Mode Interface

- Introduces address decoding.
- Fixed 4-bit burst length.
- Inflexible for asymmetric sensors.

Iteration 3: Scalable Architecture (Final)

- Supports heterogeneous sensors.
- Allows completely different input/output widths.
- Dynamic packet handling.
- Fully decoupled master/sensor communication.

System Block Diagram



Clock Domain Crossing & FIFO Synchronization

- **Asynchronous domains:** Sensors sample on CLK_{sensor} while the host communicates on asynchronous SCLK.
- **Metastability Prevention:**
  - Directly sampling cross-domain registers can lead to timing violations and system failure.
  - Integrated an **Asynchronous CDC FIFO** block (16-bit width, depth 8) between domains.
  - Write port is driven by CLK_{sensor} (valid flag controls writes).
  - Read port is driven by SCLK (controller pops values synchronously).
  - Gray code counter logic guarantees safe pointer crossings.

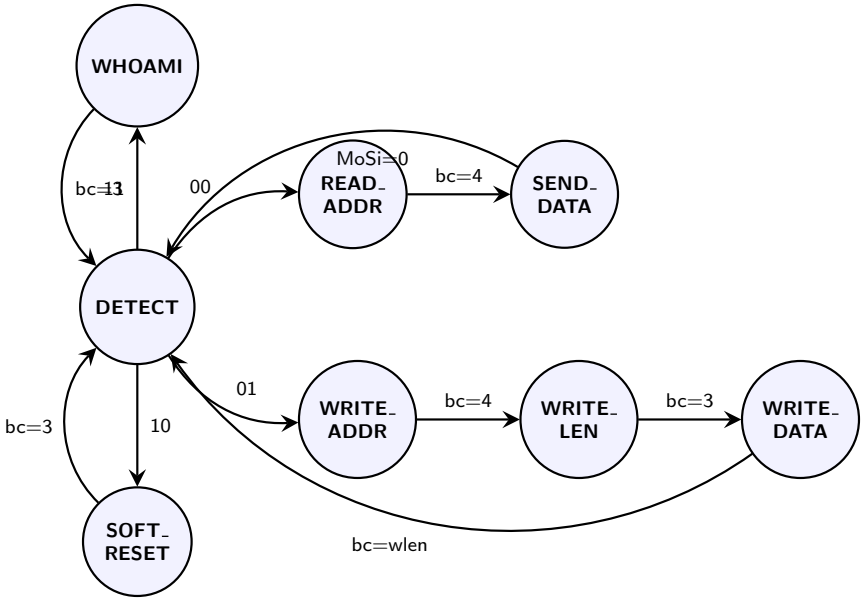
Method 1: Address + Bit-Length Protocol

- **Dynamic Packet Concept:**
  - Master controls the exact size of write transactions on the fly.
  - Sends **Mode (2 bits)**, **Address (5 bits)**, and **Length (4 bits)**, followed by data.
  - Reads are **Master-Terminated**: Master holds MOSI high to receive consecutive bits, pulls MOSI low to stop.
- **Packet Formats:**

Op	Mode (2b)	Address (5b)	Length (4b)	Data Payload
Read	00	$A_4 \ldots A_0$	N/A	Dynamic (MoSi=0 to stop)
Write	01	$A_4 \ldots A_0$	$L_3 \ldots L_0$	Variable ($L + 1$ bits)

- **Pros & Cons:**
  - *Advantage:* Highly flexible. Widths are software-defined.
  - *Disadvantage:* High transaction overhead (11 bits of control for every write).

Method 1: State Machine Diagram



Method 1: Master Write Algorithm

Master Write Operation

1: Assert CS $\leftarrow 0$
2: Shift out `Mode = 01` (Write)
3: Shift out 5-bit register address A
4: Shift out 4-bit write length L
5: **for** $i = 0$ **to** L **do**
6: Shift out payload bit on MOSI
7: Controller writes write_reg$[(A + i) \pmod{16}] \leftarrow$ MOSI
8: **end for**
9: Deassert CS $\leftarrow 1$

Method 1: Master Read Algorithm

Master Read Operation

1: Assert CS $\leftarrow 0$
2: Shift out `Mode` = `00` (Read), 5-bit address A
3: Initialize $i \leftarrow 0$
4: **while** MOSI is driven high (1) by Master **do**
5: Read MISO bit $\leftarrow$ read_reg$[(A + i) \pmod{16}]$
6: Increment $i \leftarrow i + 1$
7: **end while**
8: Deassert CS $\leftarrow 1$

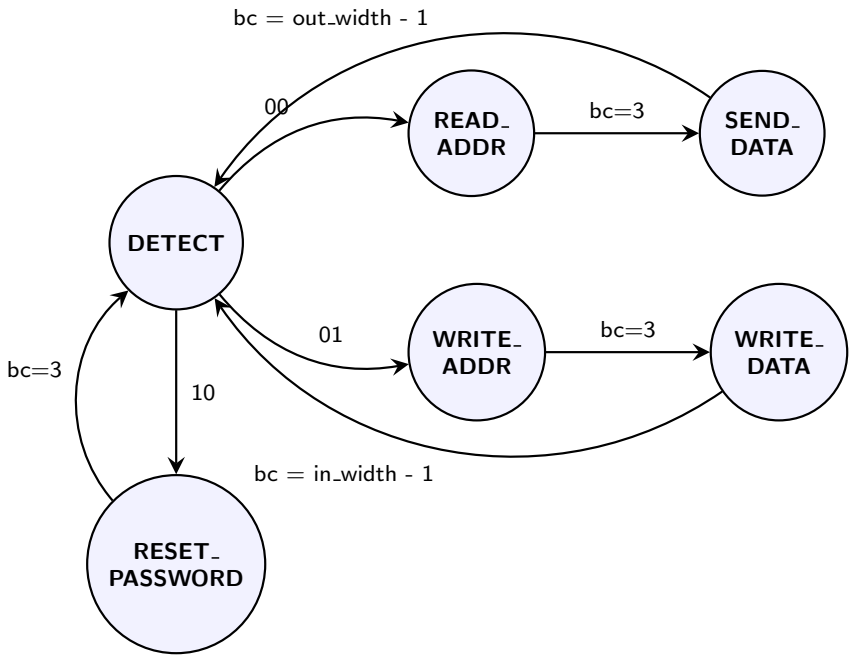
Method 2: ROM Lookup Table Protocol

- **Logic and Metadata Storage:**
  - Metadata is pre-compiled into an on-chip combinational ROM.
  - Stores: **Input Width**, **Output Width**, **Input Start Index**, **Output Start Index**.
  - The master only sends the address; the controller automatically adjusts transaction settings.
- **ROM Allocation Design:**

Address (4b)	In Width	Out Width	In Start	Out Start
4'd0	3	4	0	0
4'd1	4	4	3	5
4'd2	5	4	7	11
4'd3	8	8	8	8

- **Pros & Cons:**
  - *Advantage:* Eliminates length field packet overhead (only 6 control bits).
  - *Disadvantage:* Static. Changing configuration requires rebuilding FPGA bitstream.

Method 2: State Machine Diagram



Method 2: ROM Lookup Table Algorithm

Controller ROM Lookup & Interface Logic

Assert CS $\leftarrow 0$
Shift out `Mode` (2 bits: `00`=Read, `01`=Write)
Shift out 4-bit sensor address A
ROM Lookup:
    $\text{in_width}, \text{in_start} \leftarrow \text{ROM}[A]$
    $\text{out_width}, \text{out_start} \leftarrow \text{ROM}[A]$
if `Mode` == `01` (Write) **then**
    for $i = 0$ **to** $\text{in_width} - 1$ **do**
        Shift out payload bit on MOSI
        $\text{write_reg}[(\text{in_start} + i) \pmod{16}] \leftarrow \text{MOSI}$
    end for
else if `Mode` == `00` (Read) **then**
    for $i = 0$ **to** $\text{out_width} - 1$ **do**
        Read MISO bit $\leftarrow \text{read_reg}[(\text{out_start} + i) \pmod{16}]$
    end for
end if
Deassert CS $\leftarrow 1$

Verilog Code Snippet: slave_info_rom

```
module slave_info_rom (
    input  [3:0] addr,
    output reg [3:0] in_width,
    output reg [3:0] out_width,
    output reg [3:0] in_start,
    output reg [3:0] out_start
);
    always @(*) begin
        case (addr)
            4'd0: begin
                in_width  = 4'd3;   out_width = 4'd4;
                in_start  = 4'd0;   out_start = 4'd0;
            end
            4'd1: begin
                in_width  = 4'd4;   out_width = 4'd4;
                in_start  = 4'd3;   out_start = 4'd5;
            end
            default: begin
                in_width  = 4'd4;   out_width = 4'd4;
                in_start  = 4'd12;  out_start = 4'd12;
            end
        endcase
    end
endmodule
```

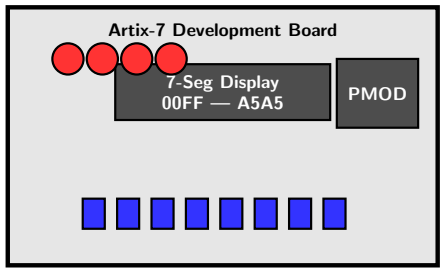
Verilog Code Optimization Details

- **Replacement of Modulo Operators:**
  - Modulo division (`% 16`) compiles to complex, high-latency division blocks during synthesis.
  - Optimized to bitwise AND masks (`& 4'b1111`) to generate instantaneous 1-cycle logic paths.
- **Single-Driver Assurance:**
  - All operations updating the registers are strictly confined to a single synchronous clock block.
  - Prevents multi-source synthesis errors during compilation.
- **MISO Tri-state Buffering:**
  - MISO is explicitly assigned to high-impedance (`1'bz`) when `cs` is high.
  - Allows sharing the physical line on a multi-peripheral bus.

FPGA Board Integration

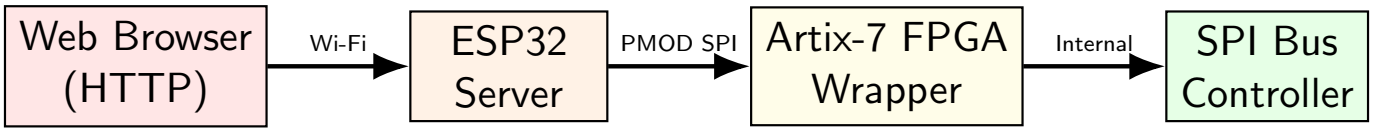
Validation Environment:

- Implemented on Xilinx Artix-7 FPGA (`xc7a35tcsg324-1`) in Vivado.
- Emulates real-time sensor inputs using 16 physical DIP switches.
- Dynamic driver multiplexes `read_reg` (hex) and `write_reg` on an 8-digit 7-segment display.
- Board status mapped to LEDs.



ESP32 Web Server Integration

- **Remote Monitoring Architecture:**
  - ESP32 micro-controller hosts a local web server (HTTP / WebSockets).
  - Provides interactive controls (Select Sensor, Write hex payload, triggering Read/Write operations).
  - ESP32 drives the FPGA pins through physical PMOD SPI headers.



Resource Utilization Results

FPGA Resource	Method 1	Method 2	Available (Artix-7)
LUTs	78	62	20,800
Flip-Flops	68	54	41,600
BRAM Blocks	0	0	50
Setup Slack	+15.24 ns	+15.24 ns	Target: 50 MHz
Max Frequency (F_{max})	142.8 MHz	166.4 MHz	—

Table: Resource and Timing Analysis Report

- **ROM Lookup Efficiency:** Method 2 shows a **20.5% reduction in LUT usage** and a **20.6% reduction in Flip-Flops** due to a smaller FSM (6 vs 9 states).

Methodology Comparison Matrix

Metric	Method 1: Address + Length	Method 2: ROM Lookup
Packet Size (Write 8b)	19 bits	14 bits
Control Overhead	11 bits (57.8% of packet)	6 bits (42.8% of packet)
Latency (Write 4b)	15 cycles (1.50 μs at 10 MHz)	10 cycles (1.00 μs at 10 MHz)
On-Chip Memory (ROM)	Zero	80 bits
Flexibility	High (software-defined)	Static (requires bitstream rebuild)
Continuous Throughput	Moderate (5.26 Mbps)	High (7.14 Mbps)
Recommended for	Software prototyping, dynamic nodes	Production systems, fixed devices

Table: Detailed Comparison of Implemented SPI Methods

- **Latency Reduction:** Method 2 provides a **33.3% improvement in transaction speed** by removing the length configuration phase.

Methodology Comparison: Pros and Cons

Method	Advantages (Pros)	Drawbacks (Cons)
Method 1: Address + Length	- Software-defined dynamic widths - No re-synthesis required for new sensors - High runtime flexibility	- Large control overhead (11 bits per write) - Higher transaction latency - Complex state machine
Method 2: ROM Lookup	- Very low control overhead (6 bits) - 33.3% faster data transfers - 20% smaller FPGA logical footprint	- Hardcoded register metadata - Changing specs requires FPGA re-synthesis - Constrained by ROM space

Table: Pros and Cons Tradeoffs

Conclusion & Future Work

- **Key Accomplishments:**
  - Successfully resolved the $O(N)$ routing footprint of standard SPI configurations.
  - Decoupled master clock phase dependencies from sensor domains using an asynchronous CDC FIFO interface.
  - Method 2 (ROM Lookup) proved to be 33.3% faster and consumed 20.5% less logic gates compared to Method 1.
- **Future Directions:**
  - **Programmable RAM Configuration Map:** Allow boot-time updates to sensor offsets, giving Method 2 the dynamic capability of Method 1.
  - **DMA Interface:** Integrate Direct Memory Access to write data blocks directly to memory, avoiding CPU cycle consumption.
  - **Bus Protocol Wrappers:** Expose standard AMBA AXI4-Lite or Wishbone wrappers for easy SoC system integration.